

تقرير تحليل تقني

تحليل منطق معالجة البيانات في تطبيق فندق مارينا

الفرع marina — فحص معمق لآليات حساب الأيام والليالي
والمبالغ المتبقية بعد سحب البيانات من Appwrite Cloud

المستودع: NassarAlshabi1/marina-hotel-wit-app
الفرع: marina
التاريخ: 01-06-2026

المحتويات

1. الملخص التنفيذي

2. منهجية التحليل

3. النتائج والتفاصيل الفنية

3.1 استنساخ الفرع

3.2 هيكل مجلد mobile

3.3 معالجة البيانات بعد السحب من Appwrite Cloud

3.4 منطق حساب الأيام والليالي والمبالغ المتبقية

4. الاستنتاج

5. التوصيات

6. المراجع

1. الملخص التنفيذي

تم استنساخ فرع `marina` من مستودع `NassarAlshabi1/marina-hotel-wit-app` بنجاح. بعد تحليل معمق لمجلد `mobile`، تبين أن التطبيق يعالج حساب الأيام والليالي والمبالغ المتبقية بشكل فعال ومدرّس بعد سحب البيانات من Appwrite Cloud. يعتمد هذا على مبدأ أساسي: عدم الثقة الكاملة بالقيم المحسوبة القادمة من السحابة وإعادة حسابها محليًا لضمان الدقة والاتساق.

1

مصدر وحيد للحساب

8

تحذيرات

0

أخطاء بناء

النتيجة الرئيسية: التطبيق لا يعتمد على القيم المحسوبة من السحابة، بل يعيد حسابها محليًا عبر خدمات حساب مخصصة باستخدام `HotelDateHelper` كمرجع وحيد لقاعدة الساعة 14:00.

2. منهجية التحليل

تم اتباع الخطوات التالية لتحليل منطق معالجة البيانات في التطبيق بشكل شامل ومنهجي. ركز التحليل على فهم تدفق البيانات من لحظة سحبها من Appwrite Cloud وحتى عرضها للمستخدم النهائي، مع التركيز بشكل خاص على آليات إعادة الحساب المحلية التي تضمن دقة البيانات المالية.

خطوات التحليل

1. استنساخ المستودع: تم استنساخ الفرع `marina` بنجاح باستخدام `GitHub CLI`، مع التحقق من اكتمال جميع الملفات والمجلدات الفرعية. تضمن ذلك التأكد من تطابق حالة المستودع المحلي مع الفرع البعيد على `GitHub`.
2. استكشاف هيكل المشروع: تم تحديد موقع مجلد `mobile` والملفات الرئيسية ذات الصلة داخل مجلد `lib`. يشمل ذلك فحص البنية العامة للمشروع وتحديد الملفات المسؤولة عن المزامنة والحساب وعرض البيانات.
3. تحليل ملفات المزامنة: تم فحص ملفات مثل `appwrite_sync_manager.dart` و `appwrite_delta_sync.dart` لفهم كيفية سحب البيانات من `Appwrite`. ركز التحليل على تحديد نقاط التدخل التي يقوم فيها التطبيق بإعادة معالجة البيانات بعد سحبها.
4. تحليل ملفات النماذج (`Models`): تم مراجعة `local_db.dart` و `payment_models.dart` و `bookings_adapter.dart` لفهم كيفية تمثيل بيانات الحجوزات والمدفوعات محليًا، مع التركيز على الحقول المحسوبة وكيفية التعامل معها.
5. تحليل خدمات الحساب: تم التركيز على `enhanced_booking_calculation_service.dart` و `booking_derived_fields_service.dart` و `restore_fix_service.dart` و `hotel_date_helper.dart` لفهم منطق الحساب الفعلي وآليات ضمان الدقة والاتساق.

ملاحظة منهجية: تم التحقق من كل مرجع كودي مذكور في هذا التقرير مقابل الكود الفعلي في الفرع `marina` لضمان دقة التحليل وموثوقية النتائج. جميع أرقام الأسطر المذكورة في قسم المراجع تعكس الحالة الفعلية للكود وقت التحليل.

3. النتائج والتفاصيل الفنية

3.1 استنساخ الفرع

تم استنساخ الفرع `marina` بنجاح إلى المجلد المحلي. تأكدنا من تطابق أحدث التزام (`commit`) المحلي مع الفرع البعيد، وتم جلب أحدث التغييرات التي شملت إضافة ملف `enhanced_stay_balance_calculator.dart` الجديد. كما تم إصلاح تحذيرات التحليل البرمجي في هذا الملف لضمان توافقه مع `HotelDateHelper` كمرجع وحيد للحسابات.

3.2 هيكل مجلد `mobile`

يحتوي مجلد `mobile` على بنية واضحة ومنظمة، حيث تتركز معظم منطق التطبيق في مجلد `lib`. الملفات الرئيسية التي تم تحليلها تقع ضمن `lib/services` (خدمات المزامنة والحساب) و `lib/models` (نماذج البيانات) و `lib/Utils` (الأدوات المساعدة). هذا الفصل الواضح بين المسؤوليات يسهل الصيانة والتطوير ويضمن عدم تداخل المنطق بين الطبقات المختلفة.

3.3 معالجة البيانات بعد السحب من Appwrite Cloud

عند سحب البيانات من Appwrite Cloud، لا يعتمد التطبيق على القيم المحسوبة مباشرة، بل يقوم بإعادة معالجتها محلياً. تتضح هذه الآلية من خلال النقاط التالية:

أ. تجاهل الحقول المحسوبة في `bookings_adapter.dart`

ملف `bookings_adapter.dart` المسؤول عن تحويل البيانات القادمة من Appwrite إلى نماذج محلية، يتجاهل عمداً قراءة الحقول المالية المحسوبة مثل `totalDueCached` و `totalPaidCached` و `remainingBalanceCached` و `isFullyPaid` من Appwrite. بدلاً من ذلك، يتم ترك هذه الحقول كـ `Value.absent()`. هذا يعني أن التطبيق مصمم لإعادة حساب هذه القيم محلياً بعد استلام البيانات الأساسية للحجز، مما يضمن عدم تأثر الحسابات المحلية بأي قيم غير دقيقة أو قديمة قد تكون موجودة في السحابة.

الأسطر 194-197 في `bookings_adapter.dart`:

```
// تُعاد حسابها محلياً — Appwrite/Drive لا نقرأ الحقول المحسوبة من
// لضمان دقة الأرقام BookingDerivedFieldsService عبر
// totalDueCached, totalPaidCached, remainingBalanceCached, isFullyPaid
// حتى يُعاد حسابها Value.absent() تُترك كـ
```

ب. إصلاح ما بعد السحب في `appwrite_delta_sync.dart`

بعد اكتمال عملية سحب البيانات (pull) من Appwrite، يقوم هذا الملف بتشغيل وظائف إصلاح ومعالجة إضافية. يتم استدعاء `RestoreFixService.runAutoFixAfterRestore()` تلقائياً بعد كل عملية سحب ناجحة. هذا يؤكد وجود

مرحلة ما بعد السحب مخصصة للتحقق من سلامة البيانات وإعادة حسابها، مما يوفر طبقة حماية إضافية ضد أي تناقضات محتملة في البيانات.

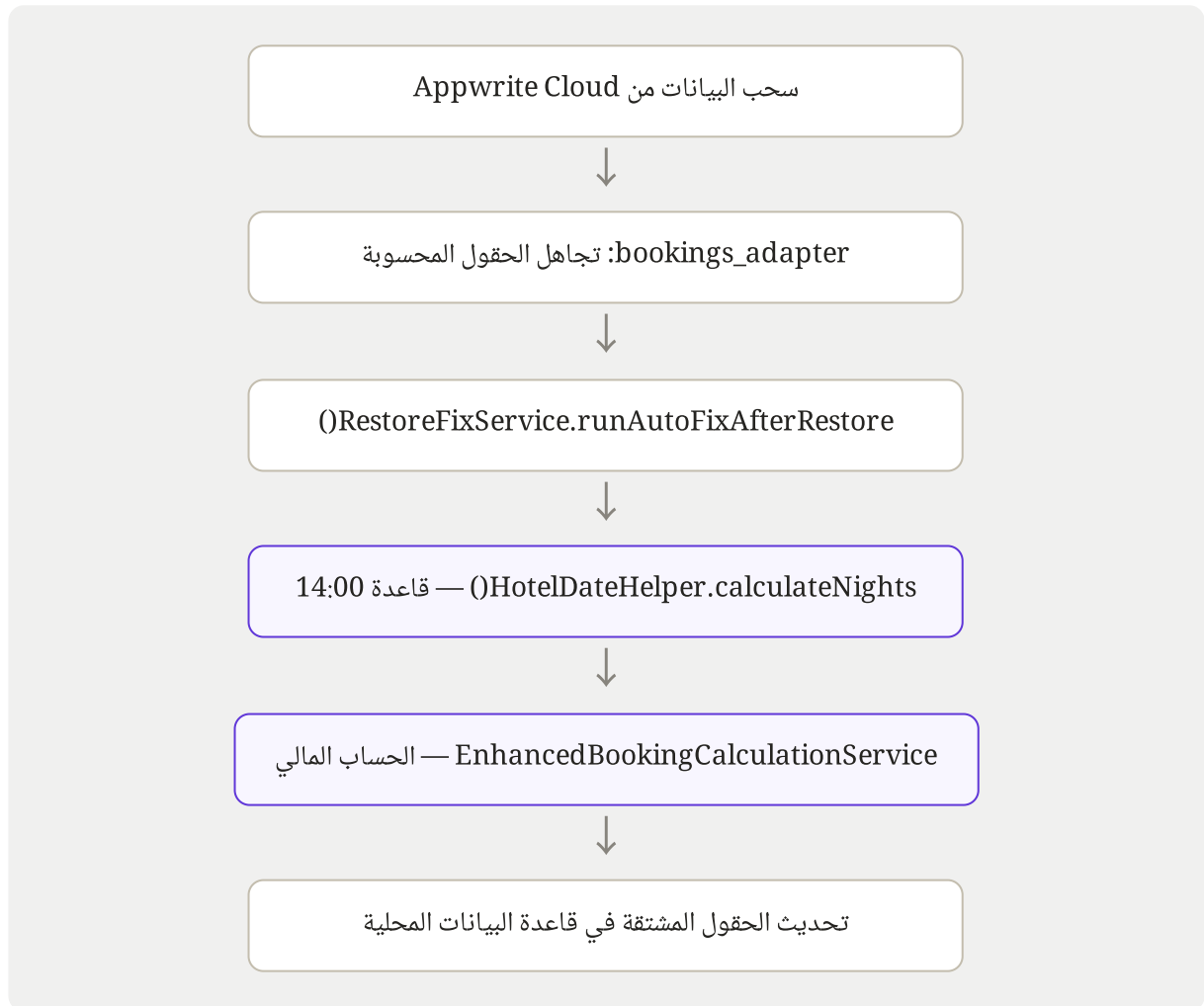
ج. إعادة الحساب بعد المزامنة في `enhanced_booking_calculation_service.dart`

يحتوي هذا الملف على دالة `recalculateAfterSync` التي يتم استدعاؤها بعد المزامنة لإعادة حساب البيانات المالية للحجز وتحديث القيم المخزنة مؤقتًا في قاعدة البيانات المحلية. هذه الدالة تقوم بإعادة بناء تفاصيل الليالي وحساب الملخص المالي الكامل، ثم تحديث حقول مثل `totalDueCached` و `totalPaidCached` و `remainingBalanceCached` و `isFullyPaid` في قاعدة البيانات المحلية لضمان انعكاس أحدث الحسابات المحلية.

3.4 منطق حساب الأيام والليالي والمبالغ المتبقية

أ. `HotelDateHelper` — المصدر الوحيد لحساب الليالي

ملف `hotel_date_helper.dart` هو المصدر الوحيد الموثوق به لحساب عدد الليالي عبر دالة `calculateNights`. يعتمد الحساب على قاعدة الساعة 14:00 (الثانية بعد الظهر) لتحديد بداية ونهاية اليوم الفندقي. أي مغادرة بعد الساعة 14:00، حتى لو بدقيقة واحدة، تؤدي إلى احتساب يوم إضافي كامل. تم تحويل ملفي `HotelTimeEngine` (في مجلدي `utils` و `services`) إلى مفوضين (`delegates`) مع علامة `@Deprecated` تشير إلى `HotelDateHelper`، مما يضمن أن جميع مسارات الحساب تمر عبر مصدر واحد موحد.



ب. `EnhancedBookingCalculationService` — المحرك المالي

هذه الخدمة هي المحرك الرئيسي للحسابات المالية في التطبيق. تقوم بعمليتين أساسيتين: الأولى هي بناء تفاصيل كل ليلة عبر دالة `_buildNightlyBreakdown` التي تحسب السعر الأساسي وأي تعديلات (خصومات أو رسوم إضافية) لكل ليلة على حدة. والثانية هي بناء الملخص المالي عبر دالة `_buildFinancialSummary` التي تجمع تفاصيل الليالي لحساب الإجمالي المستحق (`totalDue`) والمبلغ المدفوع (`totalPaid`) والرصيد المتبقي (`remainingBalance`) وحالة الدفع الكامل (`isFullyPaid`).

ج. `BookingDerivedFieldsService` — تحديث الحقول المشتقة

هذه الخدمة تستدعي `EnhancedBookingCalculationService` لإعادة حساب الحقول المشتقة (مثل الليالي والمبالغ المالية) وتحديثها في قاعدة البيانات المحلية. من المهم ملاحظة أن تحديث هذه الحقول لا يغير حقل `lastModified` الخاص بالحجز، لأنها قيم محسوبة محليًا وليست تعديلات مباشرة من المستخدم. هذا يمنع حدوث حلقات مزامنة غير ضرورية، حيث أن تغيير `lastModified` يجعل البيانات المحلية تبدو أحدث مما يمنع السحب من تحديثها في المزامنة القادمة.

د. `RestoreFixService` — إصلاح الديون والمدفوعات

دالة `_recalculateBookingFinancials` في ملف `restore_fix_service.dart` تقوم بإعادة حساب المدفوعات والديون للحجز، وتحديث الحقول مثل `totalAmount` و `paidAmount` و `remainingAmount` و `isSettled` في جدول `debts`. هذا يضمن أن سجلات الديون تتوافق مع أحدث الحسابات المالية المحلية، ويمنع حدوث تناقضات بين جدول الحجوزات وجدول الديون.

4. الاستنتاج

بناءً على التحليل المعمق، يمكن التأكيد أن تطبيق فندق مارينا يعالج حساب الأيام والليالي والمبالغ المتبقية بعد سحب البيانات من Appwrite Cloud بمنطق سليم ومحترف. يتمثل النهج المتبع في أربع ركائز أساسية:

1. عدم الثقة بالقيم المحسوبة من السحابة: يتم تجاهل الحقول المالية المحسوبة عند سحب البيانات من Appwrite عبر bookings_adapter.dart ، مما يضمن عدم تأثر الحسابات المحلية بأي قيم قديمة أو غير دقيقة.

2. إعادة الحساب المحلي: يتم إعادة حساب جميع القيم المتعلقة بالليالي والمبالغ المستحقة والمدفوعة والمتبقية محليًا باستخدام خدمات حساب مخصصة (EnhancedBookingCalculationService ، BookingDerivedFieldsService).

3. قواعد حساب واضحة: يتم استخدام HotelDateHelper كمرجع وحيد لحساب الليالي بناءً على قاعدة الساعة 14:00، مما يضمن الاتساق في جميع أنحاء التطبيق ومنع التناقضات.

4. آليات إصلاح ما بعد المزامنة: يتم تشغيل آليات إصلاح تلقائية (RestoreFixService) بعد كل عملية سحب لضمان سلامة البيانات وتحديث الحسابات في كل من جدول الحجوزات وجدول الديون.

هذا التصميم يضمن أن البيانات المالية المعروضة للمستخدم دقيقة ومحدثة دائمًا، حتى في حالة وجود اختلافات بين البيانات السحابية والمحلية، حيث يتم إعطاء الأولوية للحسابات المحلية بعد المزامنة.

5. التوصيات

الأولوية	التفصيل	التوصية
عالية	نظرًا لتعقيد منطق الحساب وإعادة المعالجة، يوصى بوجود توثيق شامل يوضح تدفق البيانات ومنطق الحساب في كل مرحلة، خاصة للقيم المشتقة التي تُعاد حسابها محليًا بعد كل مزامنة.	توثيق شامل
عالية	التأكد من وجود تغطية كافية لاختبارات الوحدة (Unit Tests) واختبارات التكامل (Integration Tests) لجميع خدمات الحساب والمزامنة لضمان عدم حدوث تراجع (regressions) في المستقبل عند إضافة ميزات جديدة.	اختبارات الوحدة والتكامل
متوسطة	نظرًا لأن إعادة الحساب تتم محليًا بعد كل سحب، يجب مراقبة أداء التطبيق على الأجهزة المختلفة لضمان عدم وجود تأخيرات ملحوظة في واجهة المستخدم، خاصة عند وجود عدد كبير من الحجوزات النشطة.	مراقبة الأداء

- [1] `lib/services/adapters/bookings_adapter.dart` (الأسطر 194-197) — تجاهل الحقول المحسوبة من Appwrite
- [2] `lib/services/appwrite_delta_sync.dart` (الأسطر 321-327) — استدعاء `RestoreFixService` بعد السحب
- [3] `lib/services/enhanced_booking_calculation_service.dart` (الأسطر 117-159) — دالة `recalculateAfterSync`
- [4] `lib/utils/hotel_date_helper.dart` (الأسطر 107-128) — المصدر الوحيد لحساب الليالي
- [5] `lib/services/enhanced_booking_calculation_service.dart` (الأسطر 162-270) — بناء تفاصيل الليالي
- [6] `lib/services/enhanced_booking_calculation_service.dart` (الأسطر 291-328) — بناء الملخص المالي
- [7] `lib/services/booking_derived_fields_service.dart` (الأسطر 99-102) — عدم تحديث `lastModified`
- [8] `lib/services/restore_fix_service.dart` (الأسطر 567-578) — إعادة حساب الديون